

W13 第 4 日 · 文档级更新与删除

阶段 ③ RAG · 项目 ② projects/rag-kb/

3 件 · 完整答案版

含完整答案

距死线 115 天

今天是答案版：三件的完整代码都放出来了，手机上慢慢看、慢慢琢磨。看懂了回头在电脑上自己敲一遍 —— 看过 ≠ 会写，最终验收还是「不看答案能不能写出来」。晚上没空就明天照着做。

今天动哪几个文件 先看这张表

文件	动作	加什么 / 改什么
index_store.py	加函数	delete_source() —— 按来源删全部块 (第 1 件)
ingest.py	改函数	ingest_file() 加 replace 开关 (第 2 件)
test_delete.py	新建	验证删除 + 覆盖的测试脚本
id_practice3.py	新建	热身变体 (第 0 件, 这题不给答案, 理由见下)

不用动: loader.py · splitter.py · retriever.py · qa.py · eval_run.py

开工前必看: delete 接口 今天唯一的新 API

你已经用过 `get(where=...)` 和 `query(where=...)`。`delete` 是同一种过滤写法，只是动作从「取」变成「删」。

写法

```
result = collection.delete(where={"source": <文件名>})
```

► 我实跑过的真实输出（照着预期，别抄数字）：

```
删掉 A.md 的 2 条    → 返回 {'deleted': 2}
删不存在的来源      → 返回 {'deleted': 0}, 不报错
一个条件都不给      → ValueError: At least one of ids,
                       where, or where_document must be
                       provided in delete.
```

三个必须记住的点

- 返回的是字典 `{"deleted": <条数>}`，不是数字——要 `return` 条数得自己取 `result["deleted"]`。
- 它只按条件删，不碰其他来源——这正是「下架某份制度」要的效果。
- `where` 里的文件名必须和入库时 `metadatas` 存的完全一致（我们存的是 `path.name`）。差一个字符就是 0 条，而且不报错。

第 0 件 • 热身变体 15–30 分钟

第 0 件 count_by_source

新建 `projects/rag-kb/id_practice3.py`

需求

写 `count_by_source(chunks) -> dict[str, int]` :

- 输入 `[(<文件名>, <正文>), ...]` 元组列表;
- 返回**每个来源各自的块数**, 形如 `{"<文件名A>": <数字>, "<文件名B>": <数字>}`;
- **硬性要求: 连续调用两次, 两次结果必须完全相同** (不得跨调用累积)。

验收

1. 同一输入连调两次, 打印出来一模一样;
2. 空列表 $\rightarrow \{\}$;
3. 同一来源出现多次 $\rightarrow$ 计数累加到同一个 key, 不新增 key。

完整答案 (先自己试, 卡住再看)

```
def count_by_source(chunks: list[tuple[str, str]]) -> dict[str, int]:
    # 容器写在函数体里 —— 每次调用都是新的, 不会跨调用累积
    counts: dict[str, int] = {}
    for source, _text in chunks:
        # .get(key, 0) = 拿不到就当作 0, 再 +1 存回去
        counts[source] = counts.get(source, 0) + 1
    return counts
```

逐行讲一遍:

1. `chunks: list[tuple[str, str]]` —— 输入是「元组列表」, 每个元组是 (文件名, 正文)。这里只关心文件名, 所以正文用 `_text` 表示「我不用它」(下划线开头是约定: 故意不用)。
2. `counts = {}` —— **写在函数体里**, 这是本题的关键。写在函数外面就变成全局变量, 两次调用会共用同一个字典, 第二次就接着第一次的数往下加 —— 正是你 10-08 踩的坑。

3. `counts.get(source, 0) + 1` —— 如果这个来源第一次出现, `.get()` 返回默认值 `0`, 加 1 变成 1; 第二次出现返回 1, 加 1 变成 2。一个 `.get()` 就替掉了 `if source in counts:` 的分支。
4. `for source, _text in chunks` —— 元组列表可以直接「解包」成两个变量, 不用写 `chunk[0]`。

► 我实跑过的真实输出:

第 1 次: {'A.md': 3, 'B.md': 2}

第 2 次: {'A.md': 3, 'B.md': 2}

两次相同: True

空列表: {}

单来源: {'C.md': 1}

第 1 件 • 按来源删除 主线

第 1 件 delete_source

改 `projects/rag-kb/index_store.py` , 加在 `add_chunks` 后面

可抄骨架 (照着敲)

```
# 加在 index_store.py 的 add_chunks 后面

def delete_source(collection, source: str) -> int:
    """删掉某来源的全部块，返回删掉的条数。

    其他来源不受影响；来源不存在时返回 0（不报错）。
    """
    result = collection.delete(where={"source": source})
    return result["deleted"]
```

逐行讲一遍：

1. `def delete_source(collection, source: str) -> int:` —— 第一个参数收 `collection` (和 `add_chunks` 一样，由调用方打开好传进来)，第二个是要删的来源名，`-> int` 声明返回整数。
2. `collection.delete(where={"source": source})` —— 和 `get(where=...)` 完全同一种写法。这一步真正把库里这个来源的行删掉。
3. `result["deleted"]` —— `delete` 返回的是字典，键 `"deleted"` 里才是条数。直接 `return result` 就返回了字典，不符合契约。

验收 (每条都要把真实终端输出贴我)

1. 删后 `count()` 正好减少该来源的条数；
2. 该来源 `get(where=)` 和 `query` 都不再返回它；
3. 其他来源一条不少 —— 逐来源点数，不只看总数；
4. 另起一个进程重开后仍不复活（证明删的是持久库，不是内存副本）；
5. 删不存在的来源 → 返回 0，不崩；
6. 删完再从原文件 `ingest_file` 一次，能重新入库且不残留。

第③条为什么强调：只看总数会骗你——误删了别的来源、又没删干净自己的，总数可能刚好对上。**必须按来源逐个点数。**

第 2 件 · 同来源覆盖式重入 主线 · 今天的核心

第 2 件 重新入库 = 先清旧块，再写新块

改 `projects/rag-kb/ingest.py`

先看真实证据：不删旧块会怎样

我实跑了一遍：库里有 旧版.md 的 5 块，直接 `add` 一个只有 2 块的新版本

► 我实跑过的真实输出：

```
起始 count: 8

--- 只 add 新版（5块→2块，不删旧） ---
add 没抛错
count 还是: 8
旧版.md 读回: ['旧1', '旧2', '旧3', '旧4', '旧5']
→ 新块没写进去、旧尾块 3/4/5 全在
```

这才是真正的坑：ID 撞车时 `add` 不报错，直接静默跳过 —— 你以为更新成功了，其实库一个字符都没变，新内容根本没进去，旧尾块全留着。

注意：你自己写的 `add_chunks` 会在更早一步就 `raise ValueError` 拦住你（10-06 加的重复保护）。所以覆盖必须先删、后加，顺序反了就直接撞保护。

数据流（先看懂再动手）

```
ingest_file(path, index_path, replace=False)
  ① load_document          → source + text
  ② split_text(500, 50)    → 新块
  ③ 逐块 embed             → 付费远程调用，先报量
  ④ open_index             → 打开库
  ⑤ 【今天新增】replace=True → 先 delete_source()
  ⑥ add_chunks(...)        → 写新块
```

可抄骨架（照着敲）

```
# ingest.py — 只改动这两处，其余不动
from index_store import add_chunks, delete_source, open_index

# ① 函数签名加一个开关，默认 False = 老行为不变
```

```
def ingest_file(path, index_path, replace=False) -> int:
    file = load_document(path)
    chunks = split_text(file["text"], CHUNK_SIZE, OVERLAP)
    if len(chunks) == 0:
        raise ValueError(f"文件{file['source']}: chunks块为0")

    embeddings = []
    for chunk in chunks:
        embeddings.append(embed(chunk))

    collection = open_index(index_path)

    # ② 新增: 要覆盖就先删掉这个来源的旧块
    if replace:
        delete_source(collection, file["source"])

    add_count = add_chunks(
        collection, file["source"], chunks, embeddings
    )
    return add_count
```

逐行讲一遍:

1. `replace=False` —— 默认值保持 `False`, 老调用方 (`test_ingest.py` 那三个子命令) 行为一个字不变, 不会因为今天的改动回归。
2. 为什么先算 `embeddings` 再删? —— 删之前先把新块的向量算好。如果反过来「先删、再算向量」, 算向量那一步失败了, 你库里的旧数据已经没了、新的又没进去, 这份来源就凭空消失了。
3. `if replace:` —— 只有明确要求覆盖才删。不传就还是老逻辑 (撞重复保护报错)。
4. `delete_source(collection, file["source"])` —— 复用你刚写的函数。这一步执行完, 库里该来源为空, 所以下一步 `add_chunks` 的重复检查不会触发。

► 我按上面的骨架实跑 (打桩, 不花钱) —— 长版 7 块 → 短版覆盖 2 块:

```
长版写入 7 块 | count=7
ids: ['制度.md-1' ... '制度.md-7']
短版覆盖写入 2 块 | count=2
```

```
ids: ['制度.md-1', '制度.md-2']
```

```
旧尾块 制度.md-7 是否还在: False
```

不传 `replace` → `ValueError`: 目标ids已存在

| `count` 仍 = 2 (旧数据没被动过)

注意最后一行: 不传 `replace` 时, 老行为完全保留 —— 还是报「目标ids已存在」, 而且库没被动过。这就是默认值 `False` 的意义。

验收 (必须构造「新版更短」的用例)

1. 覆盖后该来源的块 = 新版切块结果, 旧尾块 ID 不再存在;
2. 其他来源条数不变;
3. 另起进程重开后状态保持;
4. `get(where=)` 读回的正文与新版切块逐条一致。

测试脚本 `test_delete.py` 怎么写 新建 · 直接可抄

照 `test_index_store.py` 的样子写。用**离线假向量**，不花钱。

```
"""W13 第 4 日：删除与覆盖式重入的测试脚本。
```

跑法（仓库根目录）：

```
uv run python projects/rag-kb/test_delete.py
"""
import shutil
from pathlib import Path

from index_store import add_chunks, delete_source, open_index

INDEX_PATH = Path("D:/Hermes/cache/scratch/w13/delete_lab")

# 离线假向量：只验数据流，不花 API
V = [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6],
      [0.7, 0.8], [0.9, 1.0], [1.1, 1.2]]

def show(collection, label):
    print(f"{label} | count={collection.count()} "
          f"| ids={sorted(collection.get()['ids'])}")

def main():
    shutil.rmtree(INDEX_PATH, ignore_errors=True)
    c = open_index(INDEX_PATH)

    # 打底：两份来源
    add_chunks(c, "<文件A>", ["a1", "a2", "a3"], V[:3])
    add_chunks(c, "<文件B>", ["b1", "b2"], V[:2])
    show(c, "打底后")

    # ① 按来源删 <文件A>
    n = delete_source(c, "<文件A>")
    print(f"delete_source 返回：{n}")
    show(c, "删 <文件A> 后")

    # ② 其他来源点数 —— 不看总数，按来源数
    b = c.get(where={"source": "<文件B>"}["ids"])
```

```

print(f"<文件B> 还剩 {len(b)} 条: {b}")

# ③ 删不存在的来源
print(f"删不存在的来源返回: {delete_source(c, '<不存在>')}")

# ④ 覆盖: 先入长版, 再用短版覆盖
add_chunks(c, "<文件C>", ["c1", "c2", "c3", "c4"], V[:4])
show(c, "入长版 <文件C> 后")
delete_source(c, "<文件C>")
add_chunks(c, "<文件C>", ["c1", "c2"], V[:2])
show(c, "短版覆盖 <文件C> 后")
print("<文件C> 读回:",
      c.get(where={"source": "<文件C>"}["documents"]))

if __name__ == "__main__":
    main()

```

尖括号是你的: 上面 <文件A> / <文件B> / <文件C> / <不存在> 换成你自己起的文件名, 别原样抄。

要贴回来的输出

1. 每步的 `count` 和 `ids` ;
2. `delete_source` 返回的条数;
3. 覆盖后 <文件C> 读回的正文 —— 必须是 `['c1', 'c2']`, 不能有 `c3/c4`。

另外: 另起进程验证持久化

第 1 件验收第 ④ 条要求「重开不复活」。做法: 在上面脚本最后加一个 `read` 分支, 跑第二遍只看不写 —— 照 `test_index_store.py` 的 `do_read` 抄。

怎么跑 PowerShell 7

```
# 在 D:\dev\ai-dev-journey 目录下
uv run python projects/rag-kb/test_delete.py

# 静态检查三件（收工前）
uvx --offline ruff check .
uvx --offline ruff format --check .
uvx --offline pyright --pythonpath .venv/Scripts/python.exe
projects/rag-kb
```

讲回 2 题 第 2 件做完答，口头说，不用打字

1. 为什么不能只 `add` 新块就算「更新」？—— 用上面那个真实输出说清会发生什么。
2. 覆盖式更新 与 「整库删掉重建」 各付什么代价？（提示：想想远程 embedding 是按次收费的）

加练 余力才上，不机械追加

加练 两个小尾巴

1. `loader.py:49 / 57` 冗余切片 —— `suffix` 已经是 `.md`，还写成 `suffix[-3:]`。改成 `suffix == ".md"` 更清楚。
2. `ingest_file` 里「预计 N 块」和实际写入数不一致时给一句提示（现在不一致你根本看不出来）。

收工清单 做完再往下看

- **静态三件**（上面那三条命令）全绿；
- **三行「今天学会了什么」** —— 用你自己的话，别写「学习了 xx 库」；
- **commit + push**（`feat:` / `fix:` 前缀）；
- 日报和台账我整理，你不用敲。

边界（别越界做）

- 不接 HTTP 上传接口（留下一轮）；
- 不改 `qa.py` / `eval_run.py`、不跑评估集、不重算既有分数、不重跑 W12 与前两日；
- 实验一律用 `D:/Hermes/cache/scratch/` 下的练习库，**不动 `data/` 正式语料**；
- **真实 embedding 是付费调用** —— 跑之前先报量。

2026-10-09 · W13 第 4 日 · `D:\dev\ai-dev-journey`

派单依据 `handbook/07-当前任务指令.md`「十一」· 骨架与输出均经实跑核验